

Sistema de Detección de Sentimiento en Reseñas de Usuarios (DSR-AI)

Universitat Politècnica de València
Grado en Inteligencia Artificial
Proyecto I. Introducción a la IA

Adrián A. Acosta Villegas
Sergio Garfella Pérez
Ainara Sanfélix Ruiz
Jairo E. Urdaneta Colmenares

Integrantes

Adrián A. Acosta Villegas

Entrenamiento del modelo, vectorización, backend, análisis de resultados y supervisión

Ainara Sanfélix Ruiz

Análisis exploratorio, backend, memoria, análisis de resultados y supervisión

Sergio Garfella Pérez

Interfaz gráfica, frontend, scripts con Matplotlib, memoria, análisis de resultados y supervisión

Jairo E. Urdaneta Colmenares

Frontend, scripts con R, memoria, análisis de resultados y supervisión

Herramientas utilizadas

- **Organización y generación de informes:** GitHub, Gamma, Affinity, WhatsApp
- **Aplicación:** Python principalmente con las librerías Matplotlib, Pandas, NumPy, Pandas, Google Translator, Doc2Vec, SKLearn y Streamlit.

CONTEXTO Y OBJETIVO

Meta recibe **miles de opiniones diarias**. Analizarlas manualmente es **inviable**.

Necesitamos un sistema que clasifique automáticamente cada reseña como **positiva o negativa** en una interfaz fácil de utilizar.

Realizaremos el proyecto en **Python** utilizando librerías como *Doc2Vec* para el NLP y *Streamlit* para la interfaz gráfica.

STAKEHOLDERS

¿Quién se beneficia de este sistema?



Directivos

Visión global del sentimiento para decisiones basadas en datos.



Equipos de marketing

Personalización de campañas según la emoción del cliente.

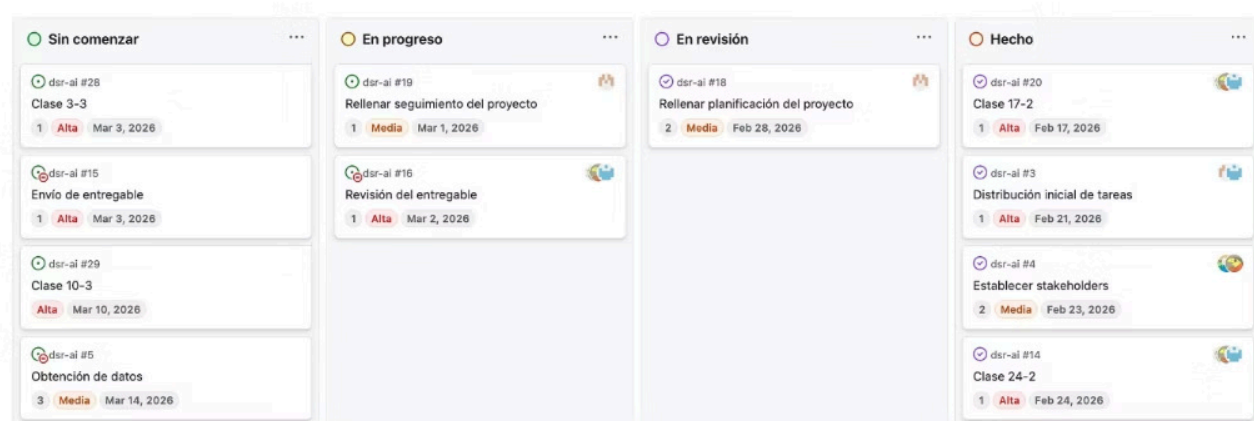


Equipos de producto

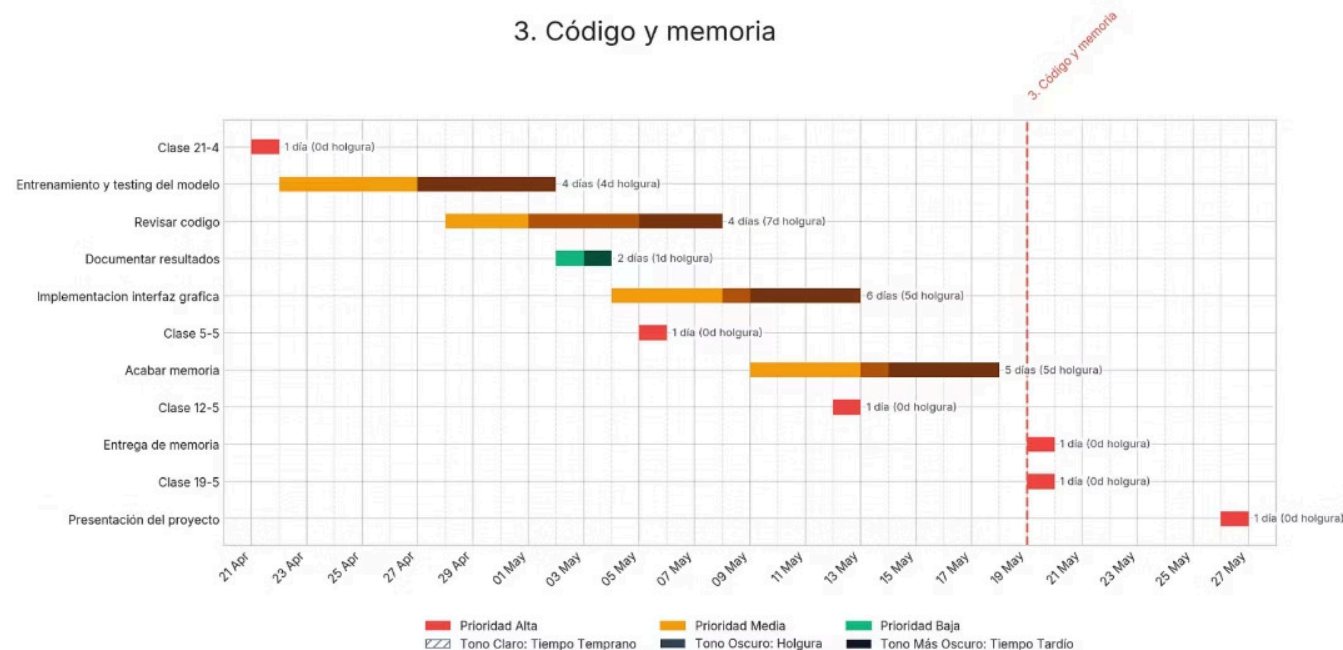
Detección de áreas de mejora a partir de feedback negativo.

PLANIFICACIÓN Y ORGANIZACIÓN

1. Definición de proyecto



3. Código y memoria



Tres datasets de referencia, más de 60.000 registros

Stanford Sentiment Treebank

De Universidad de Stanford

5 clases (transformación a binaria)

Cornell Movie Review

De Universidad de Cornell

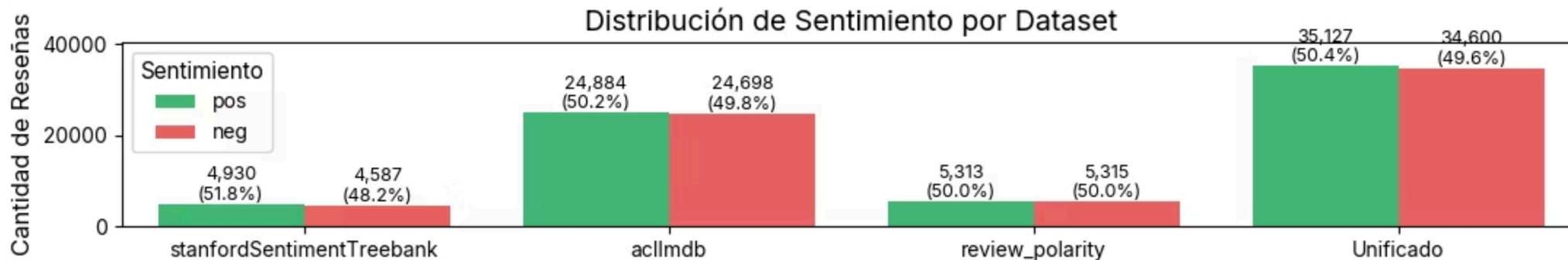
Clasificación binaria

IMDb Large Movie Review

De Universidad de Stanford

Clasificación binaria

Unificación de los datasets



Análisis Univariante y Multivariante



Longitud

Acllmdb aporta textos largos (mediana 173 palabras).

StanfordSST y *review_polarity* concentran textos cortos (mediana 19-20).

A través del Test de Mann-Whitney U comprobamos que **la longitud no es un predictor del sentimiento**. (confianza al 99%)



Ley de Zipf y Vocabulario

Vocabulario sesgado por *Acllmdb*.

61% son palabras únicas (hapax legomena) significa ruido.



Log-Odds Ratio y V de Crámer

LOR: probabilidad de que una palabra aparezca en las reseñas positivas en comparación con las negativas.

V de crámer: impacto o peso real tiene esta palabra para discriminar el sentimiento final.

Las palabras individuales tienen un **impacto nulo** en la clasificación del sentimiento.

Tokens	Types	Ratio TTR	Hapax Legomena	Zipf
11 878 256	394 370	0.0332	239,624 (60.8%)	-1.204

Evaluación de Supuestos y Outliers

Supuestos Estadísticos



No Normalidad

Test de Shapiro-Wilk rechaza normalidad ($p < 0.01$).

k -NN no asume normalidad de los datos.

Las normalizaremos por la fuerte asimetría y dispersión



Heterocedasticidad

Presencia de varianza heterogénea en las variables de longitud, vocabulario y número de caracteres.

Gestión de Outliers



IQR

Uso de Rango Intercuartílico para winsorización, evitando sesgos por asimetría extrema.



Reseñas Extremas

Identificamos 417 reseñas muy cortas (< 5 palabras) y 695 muy largas ($> \text{Percentil } 99$).



Nuestro dataset no tiene valores ausentes. Ha sido un problema que no hemos tenido que tratar.

Conclusiones del Análisis



Buen equilibrio de clases

Las clases positiva y negativa están equilibradas



Tratamiento de Outliers

Limpieza estructural eliminando registros sin contexto o con un ruido extremo.

Truncaremos las reseñas siguiendo los límites del rango intercuartílico.



Limpieza Semántica

Conversión a minúsculas.

Eliminación de stopwords (artículos/preposiciones).

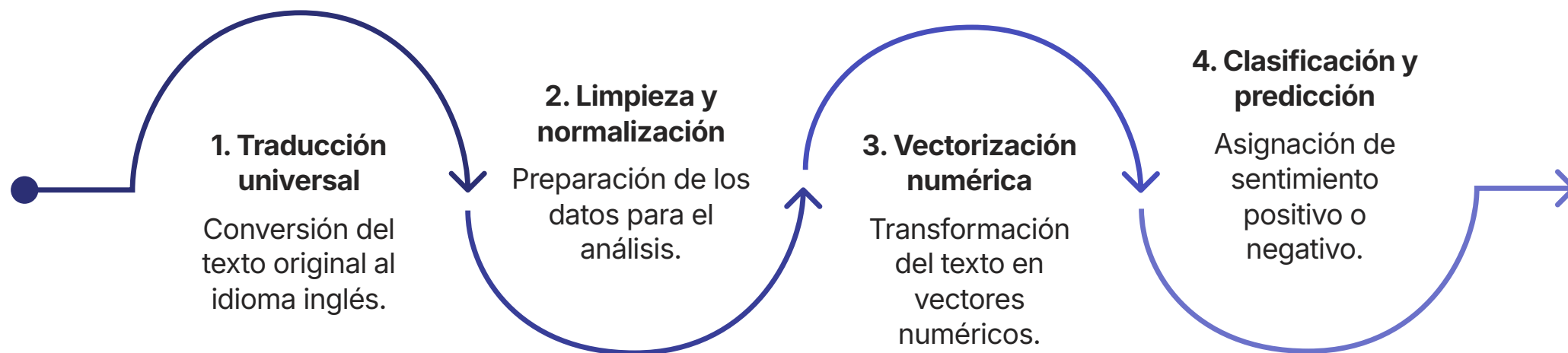
Reducir el diccionario y potenciar la señal predictiva



Doc2Vec

Utilizaremos Doc2Vec para capturar el contenido semántico esencial para la clasificación.

Flujo de procesamiento de texto

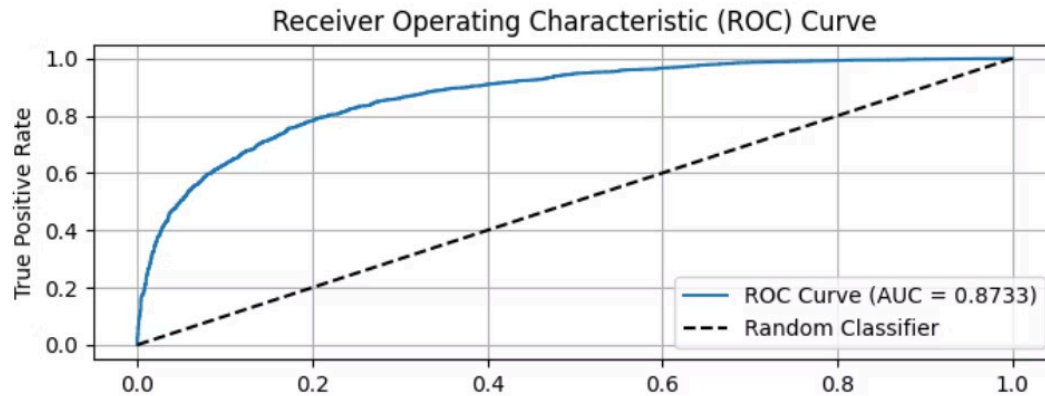


Doc2Vec no solo lee palabras: captura el **contexto matemático completo de la frase**, permitiendo al modelo entender matices semánticos que los enfoques tradicionales pasan por alto.

Dos enfoques: k-NN y Regresión Logística

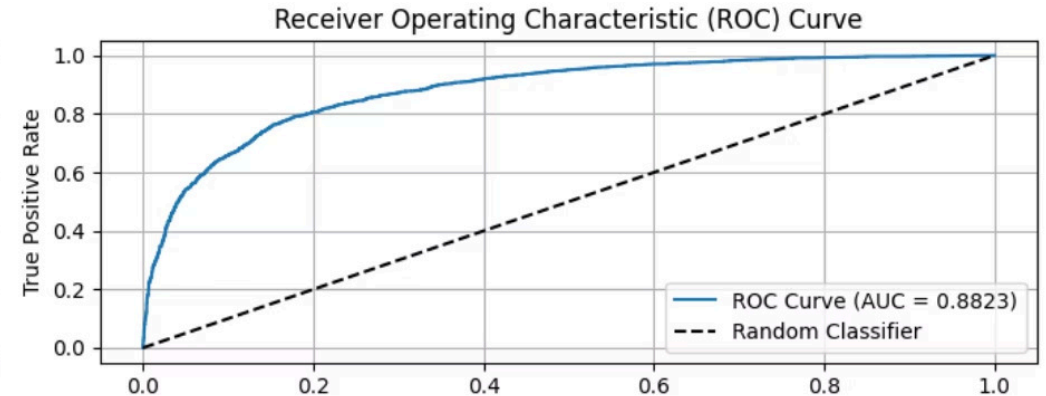
→ k-Nearest Neighbors

Clasifica según la **distancia** a los ejemplos más cercanos en el espacio vectorial.



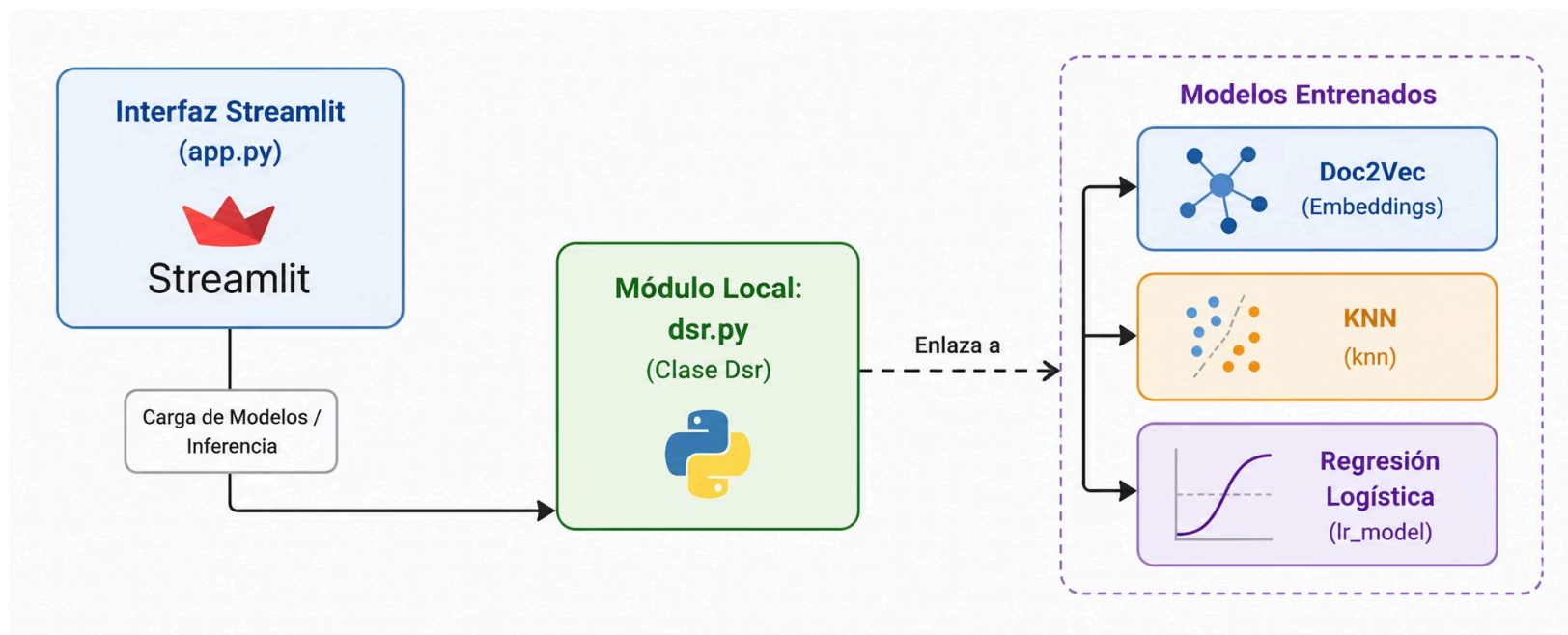
→ Regresión Logística

Calcula **probabilidades de pertenencia** a cada clase. Generaliza mejor con datos nuevos.



- ✅ La Regresión Logística superó ligeramente a k-NN con un AUC-ROC de 0,88, lo que implica un 88% de probabilidad de clasificar correctamente cualquier reseña nueva.

Interfaz Streamlit: Arquitectura del Sistema



- ✓ El software sigue una **arquitectura desacoplada**, en la que la interfaz de usuario se comunica directamente con el núcleo lógico predictivo empaquetado localmente

DESPLIEGUE

Análisis de la interfaz gráfica Streamlit

→ Capa de Estilo y Personalización (CSS)

Fondo oscuro, suavizado de bordes, transparencias

→ Flujo del trabajo y historial

Persistencia de datos en `st.session_state`

→ Selección de modelos

K-nn o Regresión Logística

Bienvenido

[Acerca de](#)

¿Qué reseña vamos a analizar hoy?

Escribe o pega tu reseña aquí. Puedes escribir en cualquier idioma...

KNN (K-Nearest Neighbors)



Analizar

Proyecto realizado por Adrián A. Acosta Villegas, Sergio Garfella Pérez, Ainara Sanfélix Ruiz, Jairo E. Urdaneta Colmenares.

DESPLIEGUE

Despliegue de la aplicación

El sistema está empaquetado en un contenedor **Docker**, listo para desplegarse en cualquier entorno: Windows, Linux o nube.

Paquete de la app

- data/
- dsr/
- web-app/
- requirements.txt

Docker

- Utilizamos Python 3.10 debido a *Gensim*
- Archivo `.tar`
- *DockerHub*



Demo

Gracias